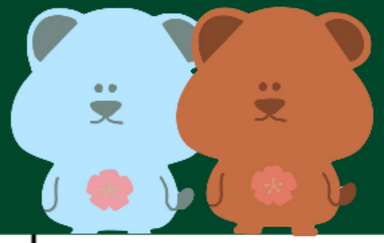
LoRA



Low-Rank Adaptation of Large Language Models

김태희

EWHA EURON

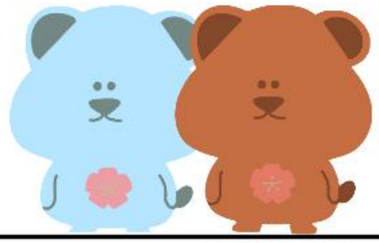
목차

#01 Introduction

#02 Method

#03 Experiments

#04 Analysis & Discussion



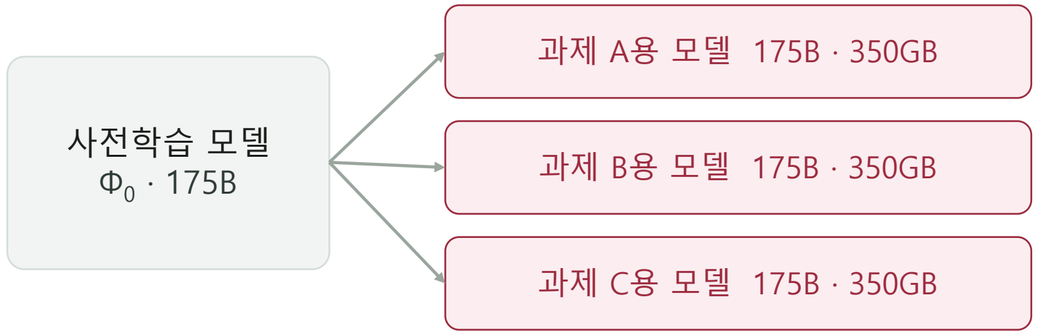
EWHA EURON

01. Introduction



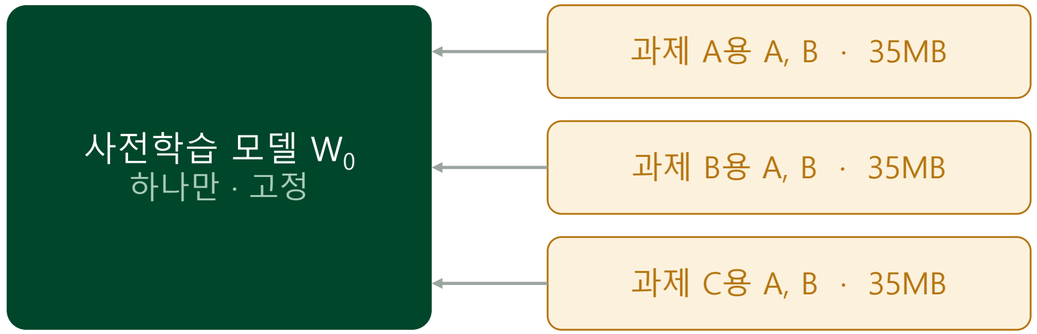
01 Introduction· full fine-tuning은 너무 비싸다

Full fine-tuning 과제마다 모델 한 벌



$|\Delta\Phi| = |\Phi_0|$ → 과제 수만큼 175B 모델을 따로 저장·배포

LoRA 모델 하나 + 작은 모듈



W_0는 공유하고, 과제가 바뀌면 작은 모듈만 갈아 끼움

모델 전체를 다시 학습하지 않고도, full fine-tuning만큼 잘할 수 있을까?

과제 100개를 저장한다면 GPT-3 175B 체크포인트 기준

Full fine-tuning ≈ 35TB (350GB × 100)

LoRA (r = 4) ≈ 354GB (350GB + 35MB × 100)

학습할 때도 무겁다

Adam은 파라미터마다 optimizer state를 들고 있어야 함

→ GPT-3 학습 VRAM 1.2TB

EWHA EURON

#01 한눈에 보는 LoRA · 결론부터

사전학습 가중치 W_0 고정, 변화량 ΔW만 작은 두 행렬의 곱 BA로 학습
→ full fine-tuning과 같은 성능을, 훨씬 적은 비용으로, 추론 지연 없이 얻었다

학습 파라미터 감소

GPT-3 175B를 Adam으로 full fine-tuning할 때와 비교. 체크포인트도 350GB → 35MB

GPU 메모리 감소

얼린 가중치는 gradient·optimizer state가 필요 없음. 학습 VRAM 1.2TB → 350GB

추론 지연 · $W = W_0 + BA$

배포 전에 BA를 W_0에 더해 두면 원래 모델과 구조가 같음 → adapter와 달리 느려지지 않음

Parameter-efficient fine-tuning | Low-rank decomposition | Intrinsic rank | No inference latency

EWHA EURON

#01 문제 정의 · 무엇을 최적화하나

Full fine-tuning 식 (1)

$$\max_{\Phi} \sum_{(x,y)\in\mathcal{Z}} \sum_{t=1}^{|y|} \log\left(P_{\Phi}(y_t \mid x,\ y_{<t})\right)$$

Φ_0에서 출발해 $\Phi_0 + \Delta\Phi$로 모든 파라미터를 갱신
$|\Delta\Phi| = |\Phi_0|$ → GPT-3라면 과제마다 175B

Parameter-efficient 식 (2)

$$\max_{\Theta} \sum_{(x,y)\in\mathcal{Z}} \sum_{t=1}^{|y|} \log\left(p_{\Phi_0+\Delta\Phi(\Theta)}(y_t \mid x,\ y_{<t})\right)$$

$\Delta\Phi = \Delta\Phi(\Theta)$를 훨씬 작은 파라미터 Θ로 표현하고 Θ만 최적화
$|\Theta| \ll |\Phi_0|$

LoRA에서 $\Theta = \{A, B\}$ → GPT-3에서 $|\Theta|$는 $|\Phi_0|$의 0.01%까지

목표는 그대로 조건부 언어 모델링

과제 데이터 $Z = \{(x_i, y_i)\}$에서 프롬프트 x가 주어졌을 때 정답 y의 확률을 최대화

요약 x 기사 → 요약문

NL2SQL x 자연어 질문 → SQL

MRC x 지문 + 질문 → 답 (기계 독해)

x, y 모두 토큰 시퀀스
제안 자체는 학습 목표와 무관 — 언어 모델링은 설명을 위한 예시

pretrained $P_\Phi(y \mid x)$: GPT 같은 범용 multi-task learner

EWHA EURON

02. Method



#02 기존 해법은 충분한가 · 추론 지연과 입력 길이 제약

Adapter Houlsby et al. (2019) · Lin et al. (2020)

추론 지연 증가율 GPT-2 medium · Fine-tune/LoRA 대비

Batch Size Sequence Length |Θ|	32 512 0.5M	16 256 11M	1 128 11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
$\text{Adapter}^{\text{L}}$	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
$\text{Adapter}^{\text{H}}$	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Table 1: Infernece latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. "|Θ|" denotes the number of trainable parameters in adapter layers. $\text{Adapter}^{\text{L}}$ and $\text{Adapter}^{\text{H}}$ are two variants of adapter tuning, which we describe in Section 5.1. The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in Appendix B.

층 사이에 끼워 넣은 adapter는 순서대로 계산해야 함 → 배치가 작은 온라인 추론에서 병렬화로 숨길 수 없음

모델을 여러 GPU에 나누면 동기화가 늘어 더 악화

Prefix tuning Li & Liang (2021)

입력 시퀀스 길이 (예: 128 토큰)



학습되는 prefix 과제에 쓸 수 있는 자리는 그만큼 줄어듦

① 최적화가 어렵다

학습 파라미터를 늘려도 성능이 단조롭게 오르지 않음

② 시퀀스 길이를 잡아먹는다

prefix가 입력 자리를 차지 → 남은 길이로 과제를 처리해야 함

기존 방법에는 속도와 품질의 제약이 있다 성능을 유지하면서 비용을 줄일 방법이 필요하다

EWHA EURON

#02 핵심 가설

① **관찰** 작은 부분공간에서도 미세조정이 가능하다

Li et al. (2018a) · Aghajanyan et al. (2020): 무작위 저차원 부분공간에 투영해도 fine-tuning이 잘 된다

② **가설** **적응할 때 생기는 변화량 ΔW도 낮은 intrinsic rank**

가중치가 '얼마나 바뀌는가'는 소수의 방향만으로 설명된다

③ **설계** **ΔW의 rank를 최대 r로 제한해서 학습**

W_0는 얼리고 ΔW = BA만 학습 — GPT-3에서 d = 12,288이어도 r = 1~2로 충분

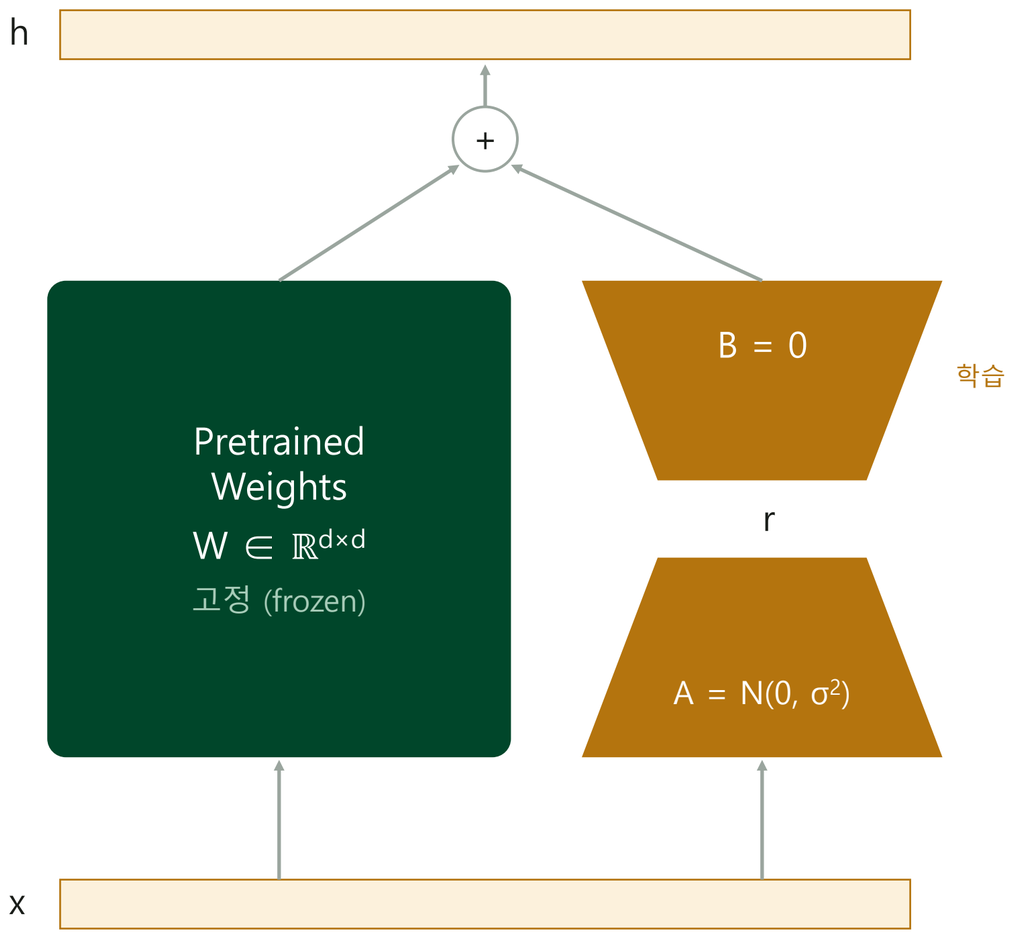


Figure 1 재구성 · 학습하는 것은 A와 B뿐

#02 LoRA를 수식으로

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

식 (3)
같은 x를 두 경로에 넣고
출력을 더함

$W_0 \in \mathbb{R}^{d \times k}$ 고정 — gradient를 받지 않음

$B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ 학습

$\Delta W = BA$ → rank가 최대 r인 보정항

초기화

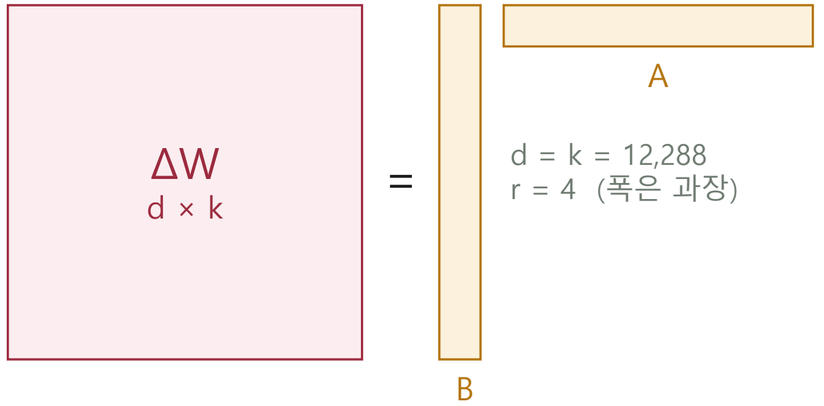
A는 랜덤 Gaussian, B는 0
→ 시작할 때 ΔW = BA = 0
→ 원래 모델 그대로 출발

스케일링 α / r

ΔWx에 α/r를 곱함
α는 처음 시도한 r 값으로 고정
→ r을 바꿔도 학습률 재튜닝 최소화

W_0는 그대로 → 학습되는 것은 rank-r 보정항 BA뿐

얼마나 줄어드나 GPT-3의 W_q 하나



full ΔW $d \cdot k = 150{,}994{,}944$

LoRA $r(d+k) = 98{,}304$

≈ 1 / 1,536

GPT-3 전체 (96층, $W_q \cdot W_v$)
r = 4 → 약 18.9M (FP16 약 35MB)
r = 1 → 4.7M (Table 4의 LoRA)

#02 추론 지연 없음

학습할 때

$h = W_0x + BAx$

W_0 고정 · A, B만 학습

배포할 때

$W = W_0 + BA$

미리 더해 두면 h = Wx → 원래 모델과 같은 연산

과제를 바꿀 때

$W - BA + B'A'$

빼고 더하기만 → 메모리 추가가 거의 없음

추론할 때 지나가는 길

Adapter: Attention | Adapter | MLP | Adapter

층이 늘어남 → 순차 계산

LoRA: Attention ($W = W_0 + BA$) | MLP

구조 그대로

full fine-tuning의 일반화

r을 원래 행렬의 rank까지 키우고 모든 가중치·bias에 적용하면
→ full fine-tuning의 표현력을 대략 복원

rank뿐 아니라 적용 범위와 bias 학습도 필요

$W = W_0 + BA$로 합쳐 두면 추론 지연 0 — fine-tuning한 모델과 구조가 같으니까 (by construction)

EWHA EURON

#02 Transformer에 적용

Transformer 한 층 d_{model} = 12,288 (GPT-3)

Self-attention

W_q	W_k	W_v	W_o
+ BA	고정	+ BA	고정

W_q(또는 W_k, W_v)는 head로 나뉘어도 $d_{model} \times d_{model}$ 행렬 하나로 취급

MLP $d_{ffn} = 4 \times d_{model}$

고정 (단순함 · 효율을 위해)

LayerNorm · bias도 고정

MLP·LayerNorm·bias에 적용하는 것은 future work

실제 이득 GPT-3 175B

VRAM	1.2TB → 350GB 최대 2/3 절감
체크포인트	350GB → 35MB r = 4, W_q·W_v · 약 10,000배
학습 속도	시간 −25% 처리량 32.5 → 43.1 tokens/s
과제 전환	A, B만 교체 VRAM의 W_0는 그대로 두고 즉석 교체

한계

A, B를 W에 병합하면 서로 다른 과제의 입력을 한 배치로 묶기 어려움

지연이 중요하지 않다면 병합하지 않고 샘플마다 모듈을 고르는 방법도 가능

왜 W_q, W_v? → §7.1 실험에서 확인

EWHA EURON

03. Experiments



EWHA EURON

#03 실험 설정

RoBERTa base / large
125M · 355M
NLU · GLUE 8개 과제

DeBERTa XXL
1.5B
NLU · GLUE

GPT-2 medium / large
355M · 774M
NLG · E2E (+ WebNLG, DART)

GPT-3
175B
WikiSQL · MNLI · SAMSum

모델 크기 →

방법	학습하는 것	학습 파라미터 수 |Θ|
FT / FT^{Top2}	전체 / 마지막 2층만 (GPT-2)	전체
BitFit	bias 벡터만	아주 적음
PreEmbed	입력에 넣은 특수 토큰의 임베딩 (prefix · infix)	$d_{model} \times (l_p + l_i)$
PreLayer	모든 층에서 특수 토큰의 activation	$L \times d_{model} \times (l_p + l_i)$
$Adapter^{H, L, P, D}$	층 사이에 넣은 bottleneck adapter	$\hat{L}_{\text{Adpt}}(2d_{\text{model}}r + r + d_{\text{model}}) + 2\hat{L}_{\text{LN}}d_{\text{model}}$
LoRA	W_q, W_v 옆에 병렬로 BA 추가	$2\hat{L}_{\text{LoRA}}d_{\text{model}}r$

#03 자연어 이해 결과

학습 파라미터 수와 Avg.를 함께 비교한다. *는 이전 연구, †는 비교 조건을 맞춘 실험이다

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

Table 2: $\text{RoBERTa}_{\text{base}}$, $\text{RoBERTa}_{\text{large}}$, and $\text{DeBERTa}_{\text{XXL}}$ with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew's correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

EWHA EURON

#03 GPT-2 생성 결과

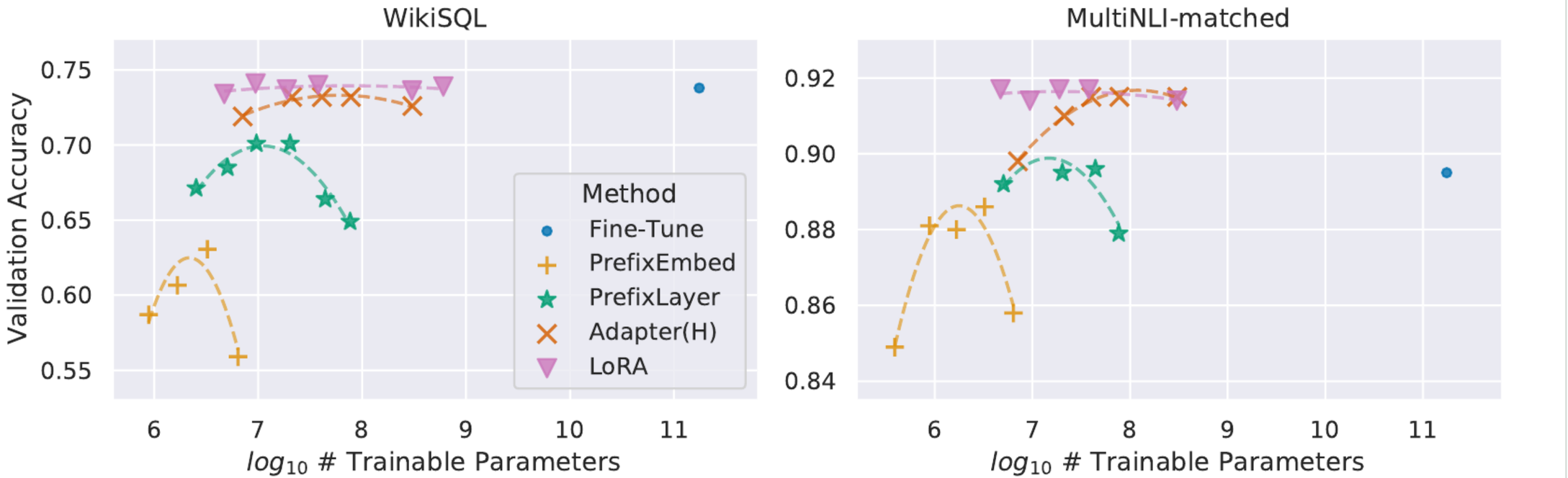
GPT-2 medium: 354.92M → 0.35M을 학습하면서 BLEU는 68.2 → 70.4

Model & Method	# Trainable Parameters	E2E NLG Challenge BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter^{L})*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter^{L})*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter^{H})	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.85}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{71.8}_{\pm.1}$	$\mathbf{2.53}_{\pm.02}$
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter^{L})	0.88M	$69.1_{\pm.1}$	$8.68_{\pm.03}$	$46.3_{\pm.0}$	$71.4_{\pm.2}$	$\mathbf{2.49}_{\pm.0}$
GPT-2 L (Adapter^{L})	23.00M	$68.9_{\pm.3}$	$8.70_{\pm.04}$	$46.1_{\pm.1}$	$71.3_{\pm.2}$	$2.45_{\pm.02}$
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.89}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{72.0}_{\pm.2}$	$2.47_{\pm.02}$

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

EWHA EURON

#03 확장성



LoRA
실험 범위에서 적은 파라미터로도 FT 수준

PreEmbed · PreLayer
특수 토큰이 256개 · 32개를 넘으면 급락 → 입력 분포가 사전학습 분포에서 멀어지는 탓으로 추정

Adapter^{H}
파라미터를 충분히 늘려야 LoRA에 가까워짐

LoRA는 예산을 바꿔도 성능이 안정적 측정한 범위에서 비교적 안정적인 성능

04. Analysis & Discussion



EWHA EURON

#04 LoRA 적용 위치

같은 18M 예산에서 Wq·Wv에 나누어 적용한 설정이 좋은 성능을 보인다

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	**73.7**	**73.7**
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	**91.7**

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

EWHA EURON

#04 rank에 따른 성능

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r. To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r. We conduct a similar experiment on GPT-2 in Section H.2.

이 두 작업에서는 Wq·Wv에 r=1을 적용해도 좋은 성능을 보인다

EWHA EURON

#04 부분공간 유사도

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U^{i\top}_{A_{r=8}} U^{j}_{A_{r=64}}\|_F^2}{\min(i,j)} \in [0,1]$$

$A_{r=8}$, $A_{r=64}$: 같은 모델에서 rank 8, 64로 따로 학습한 A
SVD로 얻은 특이벡터 중 상위 i개와 상위 j개가 이루는 공간이 얼마나 겹치나
1 = 완전히 겹침, 0 = 완전히 분리

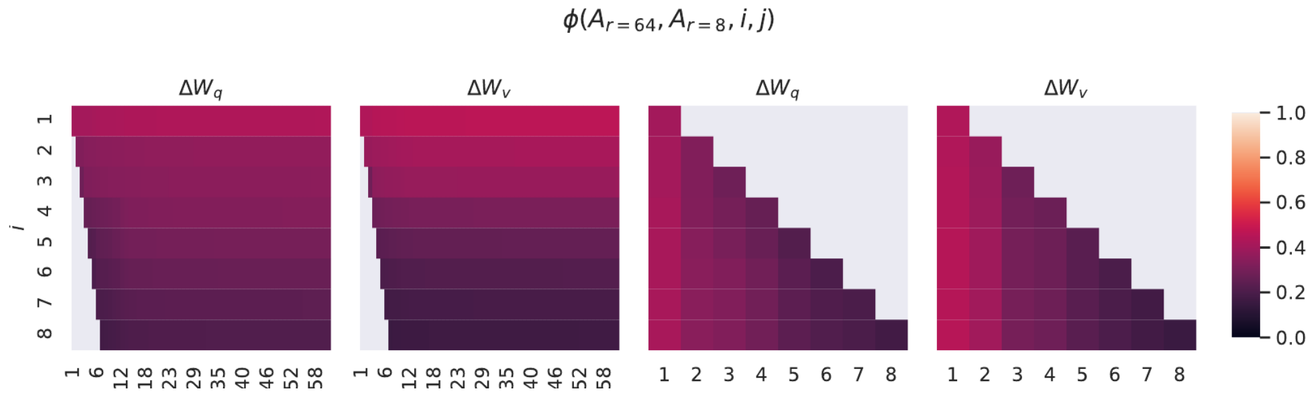


Figure 3 · GPT-3 96층 중 48번째 층 · 왼쪽 둘은 전체, 오른쪽 둘은 왼쪽 아래 삼각형을 확대 · 밝을수록 많이 겹침

맨 위 방향은 크게 겹친다
ΔW_q, ΔW_v 모두 유사도 0.5 이상 → r = 1이 잘 되는 이유

나머지 방향은 거의 안 겹친다
저자들은 노이즈일 가능성을 제시

seed만 바꿔도 (Figure 4)
ΔW_q가 ΔW_v보다 공유 방향이 많음 → rank가 더 높음
(Table 6과 일치)

쓸모 있는 정보는 소수의 상위 방향에 몰려 있다 낮은 내재적 rank 가설을 뒷받침

#04 가중치와 변화량의 관계 · Table 7

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\|\|U^\top W_q V^\top\|\|_F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\|\|W_q\|\|_F = 61.95$	$\|\|\Delta W_q\|\|_F = 6.91$			$\|\|\Delta W_q\|\|_F = 3.57$		

Table 7: The Frobenius norm of $U^\top W_q V^\top$ where U and V are the left/right top r singular vector directions of either (1) ΔW_q, (2) W_q, or (3) a random matrix. The weight matrices are taken from the 48th layer of GPT-3.

저자들은 기존 가중치에서 덜 강조된 방향을 변화량이 증폭한다고 해석한다

EWHA EURON

#04 결론과 한계

결론 · 추론 지연도, 입력 길이 손실도 없이 FT 수준의 품질 + 과제 간 빠른 전환

Transformer 언어 모델로 실험했지만, 원리는 dense layer가 있는 모든 신경망에 적용 가능

한계 · 배치에 과제 섞기

BA를 W에 병합하면 서로 다른 과제의 A, B를 한 forward로 묶기 어려움

한계 · 휴리스틱한 선택

어느 행렬에 붙일지 경험으로 정함 — 더 원리적인 방법은?

한계 · 실험 범위

MLP · LayerNorm · bias에 적용하는 것은 검증하지 않음

향후 · 다른 방법과 결합

prefix tuning 등과 직교 → 함께 쓰면 추가 개선 가능 (Appendix E)

향후 · fine-tuning의 원리

사전학습 특징이 어떻게 과제용으로 바뀌나 — FT보다 LoRA가 분석하기 쉬움

향후 · W도 low rank?

ΔW가 rank-deficient라면 W 자체도 그럴 수 있다

EWHA EURON

#04 그 이후의 LoRA

QLoRA
Dettmers et al., 2023

4-bit로 양자화한 모델 위에 LoRA → 65B 모델을 48GB GPU 한 장에서 미세조정

AdaLoRA
Zhang et al., 2023

행렬마다 같은 r 대신, 중요도에 따라 rank 예산을 배분 → '어디에 붙일까'를 학습으로

DoRA
Liu et al., 2024

가중치를 크기와 방향으로 나누고 방향에 LoRA 적용 → FT와의 차이를 줄임

LoRA+
Hayou et al., 2024

A와 B에 서로 다른 학습률을 줌 → 같은 비용으로 수렴과 성능 개선

Hugging Face PEFT
오픈소스 라이브러리

몇 줄로 LoRA를 붙이는 도구 → 오픈소스 LLM 미세조정의 기본 선택지

이미지 생성으로 확산
Stable Diffusion 등

확산 모델의 attention에 LoRA → 화풍·캐릭터를 작은 파일로 공유

W를 얼리고 저차원 보정만 학습한다 여러 후속 연구와 도구로 확장됨

EWHA EURON

#04 토의 주제 · 세션에서 함께 이야기할 것

토의 ① r = 1로 충분하다는 결과, 어디까지 믿을 수 있을까?

논문 근거 §7.2 — GPT-3의 WikiSQL·MNLI에서 W_q, W_v는 r = 1~64 사이 성능 차이가 거의 없음. 단, 각주 6에서 저자들도 "사전학습과 다른 언어의 과제라면 큰 r이 필요할 수 있다"고 단서를 달았음

생각해 볼 점 수학 추론·코드·새 언어처럼 사전학습 분포에서 먼 과제에서도 작은 rank가 통할까? 과제가 '얼마나 새로운지'를 rank로 잴 수 있을까?

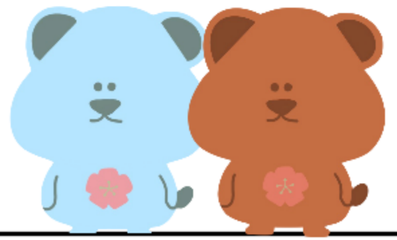
토의 ② LoRA는 full fine-tuning을 대체할 수 있을까?

논문 근거 §4.1 — r을 키우고 모든 행렬에 적용하면 FT를 대략 복원. 실험에서도 FT와 같거나 높음

후속 연구 Biderman et al. (2024) — 코드·수학을 대량으로 추가 학습하면 LoRA는 FT보다 덜 배우지만, 기존 능력은 덜 잊음. '덜 배우고 덜 잊는' 것은 단점일까, 장점일까?

EWHA EURON

THANK YOU



EWHA EURON